



PROCESAMIENTO DIGITAL DE IMÁGENES

Transformación por mapeo

Alumnos:

Enrique Ramírez Pérez, Roberto Misael Reyes Cruz

Resumen

En esta práctica se van a realizar 3 tipos de transformaciones de mapeo que nos permitan deformar de cierta manera una imagen.

En resumen, la transformación por mapeo es una técnica poderosa en el procesamiento de imágenes que permite deformar una imagen original utilizando un mapa de deformación predefinido. Esta técnica puede ser utilizada para crear efectos visuales interesantes y abrir posibilidades creativas en el procesamiento de imágenes.

Introducción

Se aplican a los píxeles de una imagen para modificar su posición, tamaño, orientación o apariencia

La transformación por mapeo es una técnica utilizada en el procesamiento de imágenes para deformar una imagen original de acuerdo con un mapa de deformación predefinido. En esta técnica, se asigna cada píxel de la imagen original a una nueva ubicación en la imagen resultante, basándose en una función de mapeo que define cómo se debe deformar la imagen.

El mapa de deformación puede generarse de diversas formas, como utilizando información de otra imagen, aplicando operaciones matemáticas o utilizando características específicas de la imagen original. Por ejemplo, en el código proporcionado, se utiliza una imagen en escala de grises como mapa de deformación, donde el valor de cada píxel de la imagen en escala de grises se utiliza para desplazar los píxeles correspondientes en la imagen original.

Al aplicar la transformación por mapeo, es posible lograr efectos interesantes y creativos en las imágenes. Dependiendo de cómo se defina el mapa de deformación, se pueden obtener

resultados como distorsiones geométricas, efectos de ondas, desplazamientos basados en características de la imagen y más.

Es importante destacar que la transformación por mapeo se basa en la manipulación de los píxeles individuales de una imagen. Esto significa que puede ser computacionalmente intensiva y requerir bucles anidados para procesar cada píxel. Sin embargo, bibliotecas como OpenCV y NumPy ofrecen herramientas eficientes para realizar estas operaciones en matrices de imágenes.

Desarrollo

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Cargar las imágenes
image_arbitrary = cv2.imread("imagencolor.jpeg")
image_gray = cv2.imread("imagengris.jpeg", cv2.IMREAD_GRAYSCALE)

# Mostrar las imágenes originales
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(image_arbitrary, cv2.COLOR_BGR2RGB))
plt.title("Imagen arbitraria")
plt.subplot(1, 2, 2)
plt.imshow(image_gray, cmap='gray')
plt.title("Imagen en escala de grises")
plt.show()

# Realizar la transformación de ondas
def apply_wave_transform(image, amplitude, wavelength):
    distorted_image = np.zeros_like(image)

    for y in range(image.shape[0]):
        for x in range(image.shape[1]):
            offset_x = int(amplitude * np.sin(2 * np.pi * y / wavelength))
            distorted_image[y, x] = image[(y + offset_x) % image.shape[0], x]

    return distorted_image

# Seleccionar valores para los parámetros de la transformación de ondas
amplitude_1 = 20
wavelength_1 = 40

# Aplicar la transformación de ondas
distorted_image_1 = apply_wave_transform(image_arbitrary, amplitude_1, wavelength_1)

# Mostrar la imagen resultante de la primera transformación de ondas
plt.imshow(cv2.cvtColor(distorted_image_1, cv2.COLOR_BGR2RGB))
plt.title("Transformación de ondas - Resultado 1")
plt.show()

# Seleccionar otros valores para los parámetros de la transformación de ondas
amplitude_2 = 40
wavelength_2 = 80

# Aplicar la segunda transformación de ondas
distorted_image_2 = apply_wave_transform(image_arbitrary, amplitude_2, wavelength_2)

# Mostrar la imagen resultante de la segunda transformación de ondas
plt.imshow(cv2.cvtColor(distorted_image_2, cv2.COLOR_BGR2RGB))
plt.title("Transformación de ondas - Resultado 2")
plt.show()
```

Se utilizará la siguiente imagen de fondo, para aplicar las tres transformaciones de mapeo. Se lee la imagen y se muestra.

```

# Realizar la transformación de apretar o pinchar
def apply_pinch_transform(image, intensity):
    distorted_image = np.zeros_like(image)

    center_x = image.shape[1] // 2
    center_y = image.shape[0] // 2

    for y in range(image.shape[0]):
        for x in range(image.shape[1]):
            dx = x - center_x
            dy = y - center_y
            distance = np.sqrt(dx ** 2 + dy ** 2)
            distortion = np.tanh(distance / intensity)
            map_x = int(center_x + distortion * dx)
            map_y = int(center_y + distortion * dy)

            if 0 <= map_x < image.shape[1] and 0 <= map_y < image.shape[0]:
                distorted_image[y, x] = image[map_y, map_x]

    return distorted_image

# Seleccionar valor para el parámetro de la transformación de apretar o pinchar
intensity = 250

# Aplicar la transformación de apretar o pinchar
distorted_image_pinch = apply_pinch_transform(image_arbitrary, intensity)

# Mostrar la imagen resultante de la transformación de apretar o pinchar
plt.imshow(cv2.cvtColor(distorted_image_pinch, cv2.COLOR_BGR2RGB))
plt.title("Transformación de apretar o pinchar")
plt.show()

# Aplicar la transformación por mapeo utilizando la imagen en escala de grises
distorted_image_mapping = np.zeros_like(image_arbitrary)

for y in range(image_arbitrary.shape[0]):
    for x in range(image_arbitrary.shape[1]):
        offset_x = int(image_gray[y, x]) - 128
        offset_y = int(image_gray[y, x]) - 128
        new_x = x + offset_x
        new_y = y + offset_y

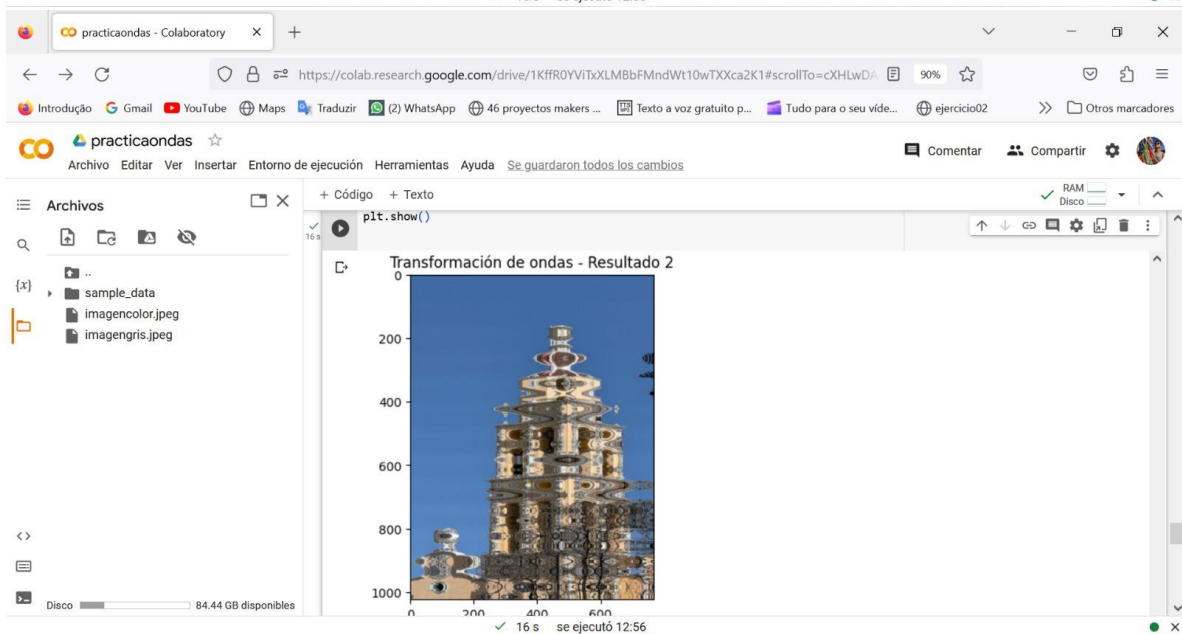
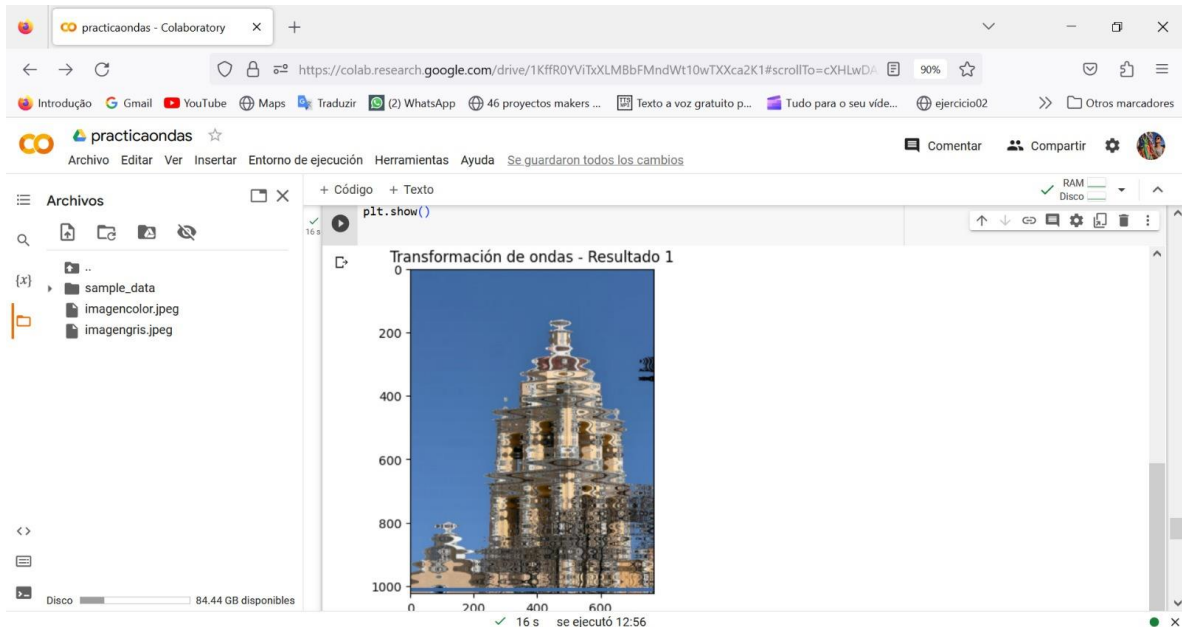
        if 0 <= new_x < image_arbitrary.shape[1] and 0 <= new_y < image_arbitrary.shape[0]:
            distorted_image_mapping[y, x] = image_arbitrary[new_y, new_x]

# Mostrar el resultado de la transformación por mapeo utilizando la imagen en escala de grises
plt.imshow(cv2.cvtColor(distorted_image_mapping, cv2.COLOR_BGR2RGB))
plt.title("Transformación por mapeo")
plt.show()

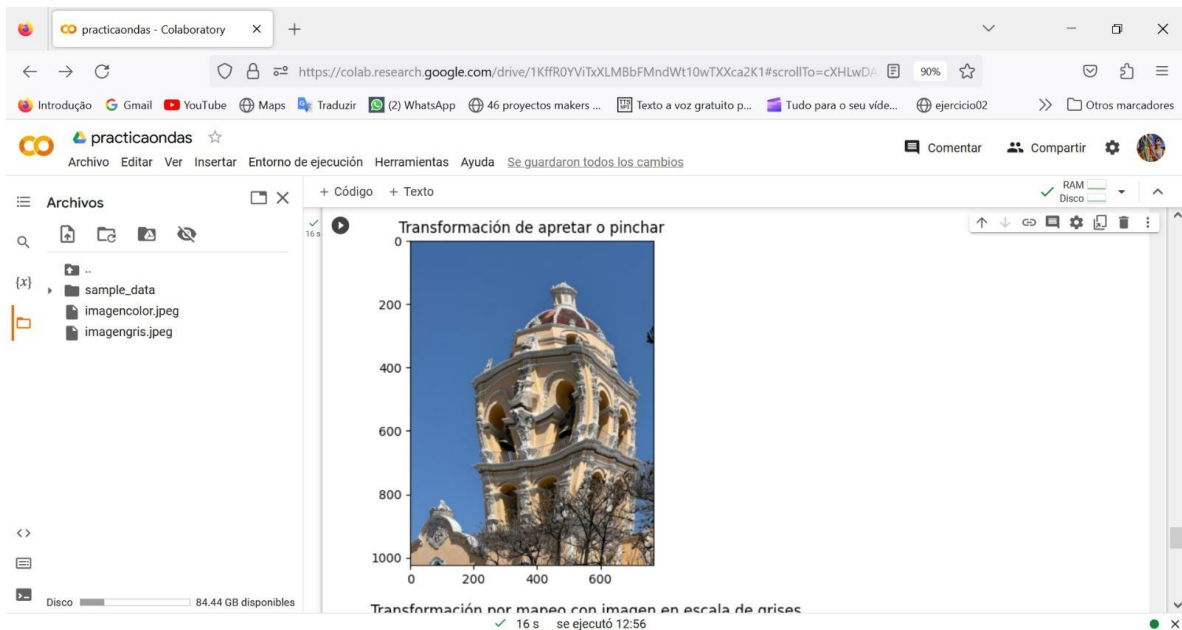
```

Esta función toma como parámetros el centro de región en donde se quiere hacer la transformación, un parámetro “gamma” que corresponde a la anchura de la zona deformada, las dimensiones de la imagen, la opción de la transformación

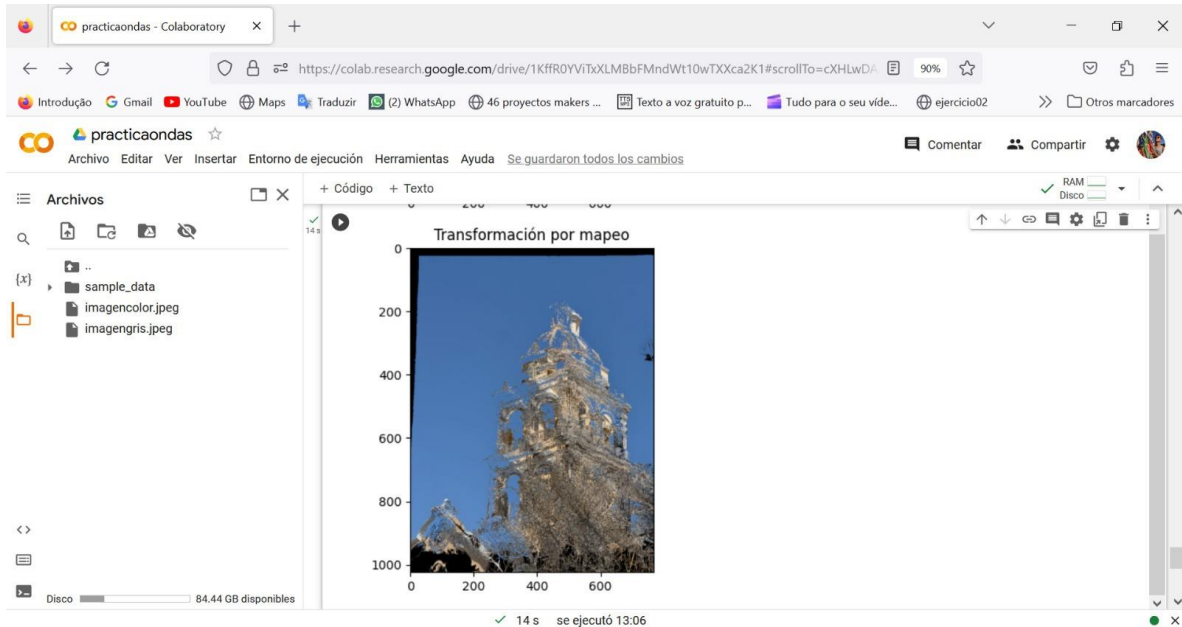
Para una superficie deformante con centro



En esta imagen se deforma (200,50), un valor de frecuencia de 0.05 y un valor de fase de 0.1, y una intensidad de transformación $a = 0.0003$



Para este caso se pasa la imagen original, las dimensiones de esa imagen, la superficie deformante, que en este caso es la imagen del conejo en escala de grises, y un grado de transformación de 0.15



Conclusión

La transformación por mapeo es una técnica utilizada en el procesamiento de imágenes para alterar la apariencia de una imagen mediante la manipulación de los píxeles de acuerdo a una función de mapeo específica. Esta técnica permite aplicar distorsiones creativas o corregir deformaciones en una imagen.

En este código, se realiza la transformación por mapeo utilizando una imagen en escala de grises como mapa de desplazamiento. Cada píxel de la imagen original se desplaza en función del valor de gris correspondiente en el mapa de desplazamiento. Esto resulta en una imagen distorsionada donde los píxeles se desplazan en diferentes direcciones y magnitudes.

La transformación por mapeo puede ser útil en varias aplicaciones, como la corrección de distorsiones en imágenes, la generación de efectos visuales y la manipulación creativa de imágenes. Al ajustar la función de mapeo, es posible lograr diferentes efectos y transformaciones en la imagen resultante.

Fuentes Bibliográficas

Gonzalez, R.C., Woods, R.E. (2008). Digital Image Processing (3rd Edition). Pearson Prentice Hall. - Este libro es una referencia ampliamente utilizada en el campo del procesamiento de imágenes y cubre temas como transformaciones geométricas y mapeo de píxeles en detalle.

Pratt, W.K. (2001). Digital Image Processing (3rd Edition). Wiley. - Este libro es otro recurso ampliamente utilizado en el procesamiento de imágenes y ofrece una cobertura completa de técnicas y algoritmos, incluidas las transformaciones geométricas.

Forsyth, D.A., Ponce, J. (2012). Computer Vision: A Modern Approach (2nd Edition). Pearson. - Este libro se centra en la visión por computadora y aborda temas relacionados con la transformación geométrica de imágenes y el mapeo de píxeles en el contexto de la visión por computadora.